

The Etruscan Tombs: Secrets of Tarquinia

Martina Leggiero
Interactive Graphics

September, 2026

1 Introduction

1.1 Project Overview

The Etruscan Tombs: Secrets of Tarquinia is an interactive first-person exploration game developed using Three.js as the final project for the Interactive Graphics course.

The application recreates a virtual environment inspired by the Necropolis of Tarquinia, one of the most significant archaeological sites of the Etruscan civilization and a UNESCO World Heritage Site. The goal of the project is not to produce an exact digital reconstruction of the necropolis, but rather to create an immersive educational experience that combines real-time computer graphics with historical and cultural content.

The player explores the archaeological site, visits reconstructed tombs decorated with frescoes inspired by the original paintings, reads historical descriptions, discovers three sacred relics, and unlocks the Sacred Chamber by placing them on the ritual pedestals.

1.2 User Experience

The application begins with a welcome screen introducing the archaeological setting and explaining the game controls.

Before entering the necropolis, the player selects a limited set of archaeological tools from an equipment menu. Once the selection is completed, an Information Sheet describing the objective of the game and a map of the necropolis are automatically added to the inventory.

The player is then free to explore the environment. Information panels positioned outside each tomb provide access to historical descriptions, while the interiors contain reconstructed frescoes and, in some cases, sacred relics required to complete the adventure.

After collecting all three relics, the player reaches the Sacred Chamber, where placing the relics on the ritual pedestals automatically opens the hidden passage leading to the sarcophagus. Interacting with the sarcophagus activates a cinematic ending, concluding the experience with an aerial view of the entire necropolis.

2 Development Environment

2.1 Software Environment

The project was developed entirely in JavaScript (ES6 Modules) using Three.js, a high-level JavaScript library built on top of WebGL for real-time 3D graphics.

Development was carried out using Visual Studio Code as the integrated development environment (IDE), while Vite was used as the local development server and module

bundler. Vite provides fast compilation, automatic module management, and live reloading, making the development process more efficient.

Version control was managed through Git, and the complete source code was maintained in a GitHub repository throughout the development of the project.

2.2 Project Organization

The source code follows a modular organization, where each subsystem is implemented in an independent JavaScript module.

The `main.js` file is responsible for initializing the application, creating the scene, configuring the rendering pipeline, managing user input, and updating the game state during the rendering loop.

The remaining modules are dedicated to specific functionalities, including world generation, environment creation, collision handling, inventory management, tomb construction, document visualization and Sacred Chamber logic.

This modular structure improves code readability, facilitates maintenance, and allows new components to be integrated with minimal changes to the existing implementation.

3 Libraries and External Assets

The project was developed using a limited number of external libraries and assets. Whenever possible, architectural structures, environmental elements, and gameplay mechanics were implemented directly using Three.js geometries and custom JavaScript code.

3.1 External Libraries

The following external libraries were used during the development of the project.

THREE.JS

Three.js is the core rendering library of the application. It provides the rendering engine and the main classes used to create and manage the 3D environment, including scenes, cameras, geometries, materials, textures, lights, shadows, and animations.

POINTERLOCKCONTROLS

The PointerLockControls module provided by Three.js was used to implement first-person navigation. It captures the mouse pointer, allowing the player to freely look around while moving through the environment using the keyboard.

GLTFLOADER

The GLTFLoader module was used to import external 3D models in GLB format. These models were integrated into the scene and combined with custom interaction logic.

3.2 Imported 3D Models

Only a limited number of external models were used in the project.

Model	Purpose
Sarcophagus	Final object of the game. Interacting with it triggers the ending cinematic sequence.
Archaeological Tools	Collectible equipment (pickaxe, brush, trowel, rope, knife, pen, lantern, etc.) that can be selected and equipped by the player.
Sacred Relics	Three collectible relics required to unlock the Sacred Chamber.

Table 1: Imported 3D models used in the project.

3.3 Textures

Textures were used to improve the visual quality of the environment while keeping the geometric complexity relatively low.

ENVIRONMENT Grass texture applied to the terrain. Dirt path texture used for the main road crossing the necropolis. Entrance ground texture combined with a normal map to increase the perception of surface detail.

TOMB INTERIORS Reconstructed frescoes inspired by the original Etruscan paintings, applied to the interior walls of the explorable tombs.

ENVIRONMENT DETAILS Cloud texture used to generate the animated clouds. User Interface Parchment texture used for the map and historical documents displayed inside the Document Viewer.

Whenever possible, textures were repeated using UV scaling in order to efficiently cover large surfaces such as the terrain and the main path.

3.4 Original Components

Although some assets were imported, most of the project was developed specifically for this application.

The original components include:

- the complete necropolis layout;
- the terrain and road system;
- all tomb entrances and interiors;
- the Sacred Chamber;
- procedurally generated cypress trees;
- procedurally generated animated clouds;
- information panel supports;
- the inventory system;
- the document viewer;
- the equipment selection interface;
- the relic placement system;
- the collision management system;
- the interaction system;
- sliding door animations;

- the final cinematic sequence.

Most architectural structures were modeled procedurally using standard Three.js geometries, while gameplay mechanics, animations, and user interface components were entirely implemented in JavaScript.

4 Graphics and Scene Design

4.1 Scene Organization

The virtual environment is organized using the scene graph provided by Three.js. Every major element of the application is represented as an independent `THREE.Group`, allowing complex structures to be assembled from multiple meshes while behaving as a single object.

This hierarchical organization simplifies scene management, object transformations, and code reuse. For example, each tomb entrance is composed of walls, roof, door, decorative elements, and an information panel grouped together into a single object. The same approach is adopted for the tomb interiors, the Sacred Chamber, and several environmental objects.

4.2 Procedural Geometry

Most of the environment was modeled procedurally using the standard geometries provided by Three.js instead of relying on imported meshes.

The following geometries were extensively used throughout the project:

- `PlaneGeometry` for the terrain, path, frescoes, clouds, and interface elements.
- `BoxGeometry` for walls, doors, roofs, pedestals, floors, and most architectural components.
- `CylinderGeometry` for columns, handles, and tree trunks.
- `ExtrudeGeometry` for the sloped lateral walls of the tomb entrances.
- `LatheGeometry` for generating the foliage of the cypress trees from a custom two-dimensional profile.

Using procedural geometries reduced the number of imported assets while allowing complete control over dimensions, proportions, and placement of each object.

4.3 Texture Mapping

Textures were employed to increase the realism of the environment without increasing geometric complexity.

The terrain is covered by a repeating grass texture, while a second repeating texture represents the main archaeological path crossing the necropolis. Texture repetition was achieved by modifying the UV coordinates through the material properties, allowing relatively small images to cover large surfaces efficiently.

Inside the tombs, reconstructed frescoes inspired by the original Etruscan paintings are applied to the walls using textured planes, recreating the appearance of the historical interiors.

A normal map was also applied to the entrance ground, adding small surface details that improve depth perception without requiring additional geometry.)

4.4 Lighting and Shadows

The illumination of the scene is based on the combination of an Ambient Light and a Directional Light.

The directional light simulates sunlight and is responsible for casting dynamic shadows throughout the environment. Both `castShadow` and `receiveShadow` were enabled for the main architectural structures, imported models, and environmental objects, increasing the overall realism of the scene.

The ambient light provides indirect illumination and prevents excessively dark areas while preserving the visibility of the environment.

A dedicated lighting animation is performed during the final cinematic sequence, where the ambient illumination gradually increases to symbolically reveal the entire archaeological site after the Sacred Chamber has been unlocked.

4.5 Camera and Navigation

The application uses a `PerspectiveCamera` combined with the `PointerLockControls` module to provide a first-person exploration experience.

Player movement is controlled using the keyboard, while the mouse controls the camera orientation after the pointer has been locked inside the browser window.

During gameplay, the camera behaves as a traditional first-person camera. At the end of the adventure, however, user control is temporarily disabled and the camera performs a scripted cinematic movement, smoothly rising above the necropolis while continuously looking towards the Sacred Chamber.



Figure 1: Interior of one reconstructed Etruscan tomb.

5 User Manual

5.1 Getting Started

When the application starts, an introductory screen presents the project and the archaeological setting. From this menu, the player can begin the adventure by selecting `Enter the Necropolis`.

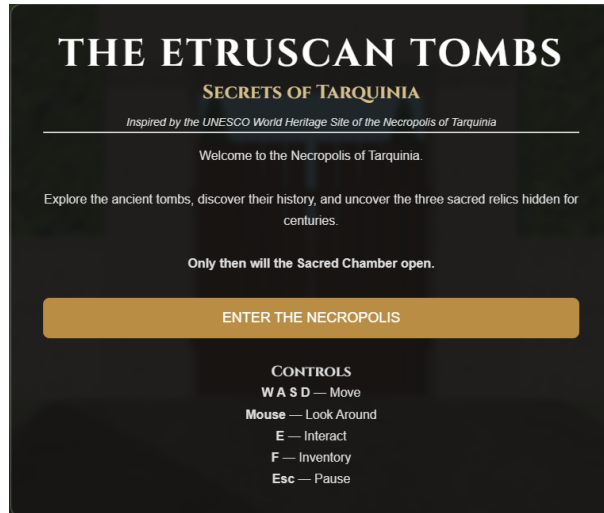


Figure 2: Initial game interface

5.2 Exploration

The environment can be freely explored from a first-person perspective. When the player approaches an interactive object, a contextual prompt appears on the screen, indicating that an action can be performed by pressing E.

The same interaction system is used to:

- enter and leave tombs;
- inspect information panels;
- collect relics;
- place relics on the pedestals;
- interact with the final sarcophagus.

5.3 Inventory

Before starting the exploration, the player interacts with an information point where a limited number of archaeological tools can be selected. After confirming the selection, an Information Sheet and a Map of the Necropolis are automatically added to the inventory. The inventory is opened by pressing F and is divided into three sections:

Equipment, displaying the selected archaeological tools;

Map, showing the layout of the necropolis;

Documents, containing all historical descriptions discovered during the exploration.

A small Equipment HUD, permanently displayed in the upper-right corner of the screen, allows the player to quickly check the currently selected equipment without interrupting gameplay.



Figure 3: Inventory interface showing the Equipment section.

5.4 Sacred Chamber and Ending

After collecting the three sacred relics, the player reaches the Sacred Chamber, where the relics can be placed on three ritual pedestals. Once all pedestals contain a relic, the hidden passage opens automatically, granting access to the final sarcophagus.

Interacting with the sarcophagus triggers the final cinematic sequence, during which the camera rises above the necropolis while the environment gradually becomes illuminated before the final completion screen is displayed.

6 Technical Implementation

6.1 Rendering Loop

The application follows the typical rendering pipeline of a real-time Three.js application.

After the scene initialization, a continuous rendering loop updates the application by processing user input, checking interactions, updating animations, and rendering the scene from the current camera position.

This approach allows all dynamic elements of the environment to be updated in real time while maintaining a smooth user experience.

6.2 Collision and Interaction System

Player movement is regulated through a custom collision management system.

Only objects that should physically block the player are registered as colliders, including architectural structures, doors, walls, trees, and other static elements. Each collider is stored in a dedicated collision list. During each frame, the player's movement is validated by testing the future position against all registered colliders before updating the camera position.

Interactive objects are handled separately. During each frame, the application computes the distance between the player and nearby interactive objects. When the player enters a predefined interaction range, a contextual prompt appears on the screen. Pressing the E key executes the corresponding action.

This mechanism is shared by all gameplay interactions, including opening tomb entrances, reading information panels, collecting relics, placing relics on pedestals, and activating the final cinematic.

6.3 Animations

All animations are generated programmatically without using predefined animation files. The implemented animations include:

- continuous cloud movement;
- wall cleaning;
- rock destruction;
- opening of the hidden passage inside the Sacred Chamber;
- gradual increase of ambient lighting during the ending;
- cinematic camera movement.

Object transformations are updated incrementally during the rendering loop, producing smooth transitions while allowing complete control over speed and timing.

6.4 User Interface Management

The graphical interface is entirely HTML-based and is dynamically controlled through JavaScript.

Different interface screens are displayed according to the current game state, including:
introductory menu;
equipment selection;
inventory;
document viewer;
relic selection menu;
pause menu;
final screen.

During exploration, contextual interface elements such as the interaction prompt, objective messages, tutorial messages, and the equipment HUD are updated dynamically according to the player's actions.

The inventory interface allows the player not only to inspect collected items but also to equip archaeological tools. Their current selection is continuously displayed through an Equipment HUD, which is updated in real time whenever a tool is equipped or unequipped, providing immediate visual feedback without interrupting exploration.

6.5 Sacred Chamber Logic

The Sacred Chamber unlocking logic is implemented independently from the rendering system.

Each pedestal stores the relic placed by the player. Once all three pedestals contain the required relics, the hidden passage is automatically opened, granting access to the final sarcophagus.

6.6 Final Cinematic

The ending sequence combines several systems developed throughout the project.

After interacting with the final sarcophagus:

- player controls are disabled;
- the mouse pointer is released;
- ambient lighting gradually increases;
- the camera smoothly moves toward a predefined aerial position using linear interpolation (lerp);

- the camera continuously faces the archaeological site while the final completion screen is displayed.

This sequence provides a cinematic conclusion to the experience while showcasing the entire reconstructed environment.

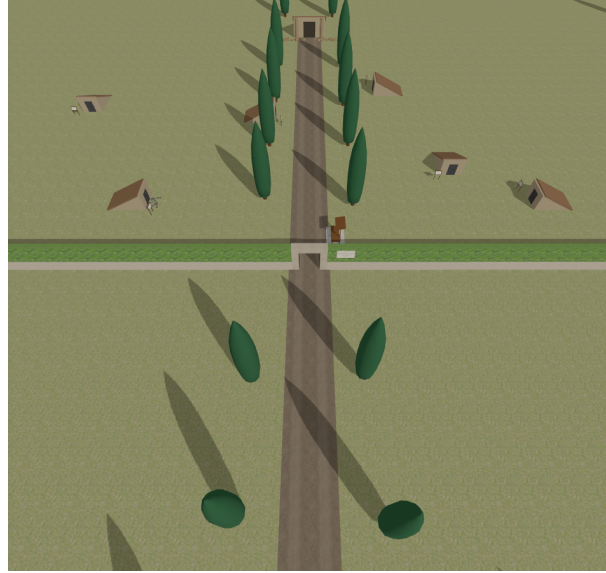


Figure 4: Final frame of the cinematic ending showing an aerial view

7 Conclusion

The project successfully combines real-time computer graphics, interactive exploration, and educational content within a single web-based application inspired by the Necropolis of Tarquinia.

From a technical perspective, the application integrates procedural modeling, texture mapping, dynamic lighting, shadow rendering, real-time animation, collision detection, first-person navigation, and modular software design using the Three.js framework.

Beyond satisfying the technical objectives of the Interactive Graphics course, the project demonstrates how modern web technologies can be employed to create an engaging educational experience that promotes the cultural heritage of one of the most important Etruscan archaeological sites.

The modular architecture adopted during the implementation also allows new gameplay mechanics and archaeological sites to be integrated with minimal modifications.

Future improvements could include environmental audio, additional explorable tombs, more advanced puzzles, dynamic weather conditions, and non-player characters to further enrich the exploration experience.